

Base de Datos Avanzada



Profesores: Ricardo Arroyo
Jorge Pérez Herrera[©]

Base de Datos Avanzada

Introducción a los índices en BD

Índices en Base de Datos

Problemática

ID	Apellido	...
1	Martínez	
2	Fernández	
3	García	
4	López	
5	Álvarez	
6	López	
7	Rodríguez	

```
select * from Personas where Apellido='García'
```

Índices en Base de Datos

Búsqueda Secuencial



ID	Apellido	...
1	Martínez	
2	Fernández	
3	García	
4	López	
5	Álvarez	
6	López	
7	Rodríguez	

```
select * from Personas where Apellido='García'
```

Índices en Base de Datos

Búsqueda Secuencial



ID	Apellido	...
1	Martínez	
2	Fernández	
3	García	
4	López	
5	Álvarez	
6	López	
7	Rodríguez	

```
select * from Personas where Apellido='López'
```

Índices en Base de Datos

¿Qué es un índice?



es una **estructura de datos** que mejora la velocidad de las operaciones de recuperación de datos en una tabla al proporcionar un acceso rápido a filas específicas.

Conceptualmente es similar al índice de un libro: permite encontrar rápidamente la información sin tener que leer todo el contenido

Índices en Base de Datos

Analogía simple

Imaginemos buscar un libro en una biblioteca

- Sin índice

Recorrer cada estante y revisar cada libro (escaneo completo de tabla).

- Con índice

Realizar búsqueda en el catálogo digital para obtener la ubicación exacta (búsqueda indexada).



Índices en Base de Datos

¿Para qué sirven?

- **Acelerar consultas:** Reducen drásticamente el tiempo de respuesta de las consultas.
- **Mejorar la eficiencia:** Reducen la cantidad de operaciones de I/O necesarias.
- **Optimizar ordenamientos:** Facilitan la obtención de datos en un orden

Índices en Base de Datos

¿Para qué sirven?

- **Garantizar unicidad:** Los índices únicos previenen duplicados en columnas clave.
- **Implementar restricciones:** Ayudan a implementar claves primarias y foráneas.

Índices en Base de Datos

Ventajas

- **Rendimiento de consultas:**
 - Búsquedas, ordenamientos y agrupaciones más rápidas.
- **Optimización de JOIN:**
 - Mejora sustancial en operaciones de unión entre tablas.
- **Eficiencia en condiciones WHERE:**
 - Particularmente en columnas frecuentemente filtradas.
- **Rendimiento en consultas de agregación:**
 - Mejora en funciones como COUNT, SUM, AVG.
- **Unicidad garantizada:**
 - En índices con restricción UNIQUE.



Índices en Base de Datos

¿Cuándo conviene un índice?

Indexar columnas que aparecen frecuentemente en
WHERE, JOIN, ORDER BY y GROUP BY.

Ejemplo con una tabla con 10 millones de filas

SIN índice

```
SELECT * FROM pedidos WHERE cliente_id = 5432;
```

✗ *Escaneo completo de la tabla:* lee 10.000.000 de filas

Con un índice en cliente_id

```
CREATE INDEX idx_pedidos_cliente ON pedidos (cliente_id);
```

```
SELECT * FROM pedidos WHERE cliente_id = 5432;
```

✓ *Búsqueda indexada:* lee solo las filas de ese cliente (ej: 47 filas)

Índices en Base de Datos

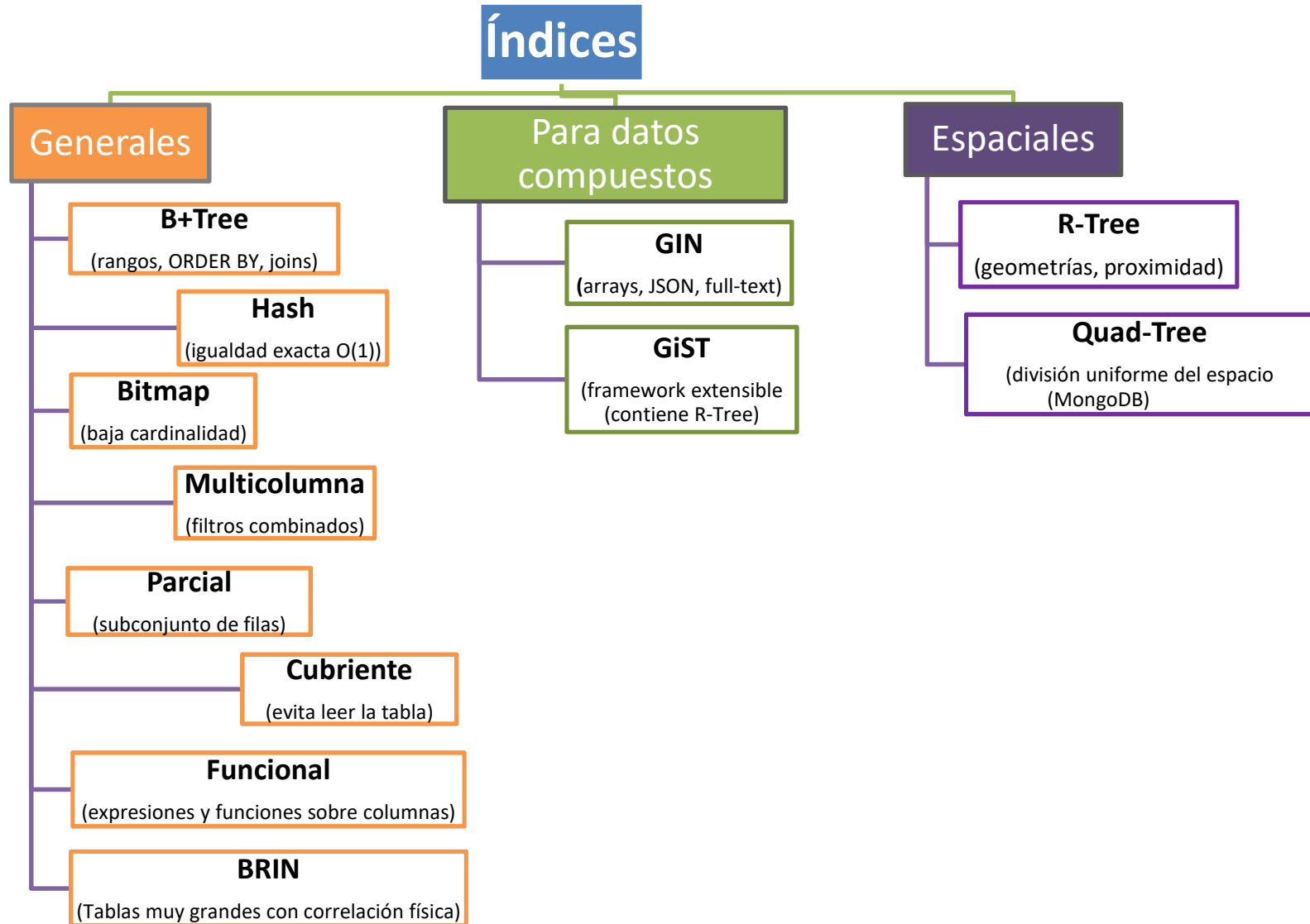
Desventajas

- **Espacio de almacenamiento:** Requieren espacio adicional (pueden ocupar tanto o más que los datos).
- **Sobrecarga en escritura:** Insertar, actualizar y eliminar registros es más costoso debido a la actualización de índices.
- **Mantenimiento:** Requieren ser reconstruidos periódicamente para optimizar su rendimiento.
- **Planificación cuidadosa:** Un exceso de índices puede degradar el rendimiento general.
- **Complejidad adicional:** Aumenta la complejidad en la administración de la base de datos.

Índices en Base de Datos

Tipos de índices

Índices en Base de Datos



Índices en Base de Datos

Índices B-Tree

Índices en Base de Datos

Tipos de índices

Índices B-Tree (Balanced Tree)

Características

- Estructura de árbol balanceado (todos los nodos hoja están a la misma distancia de la raíz)
- Optimizado para operaciones de disco (pocos accesos a disco)
- Mantiene los datos ordenados
- Soporta búsquedas de rango eficientes

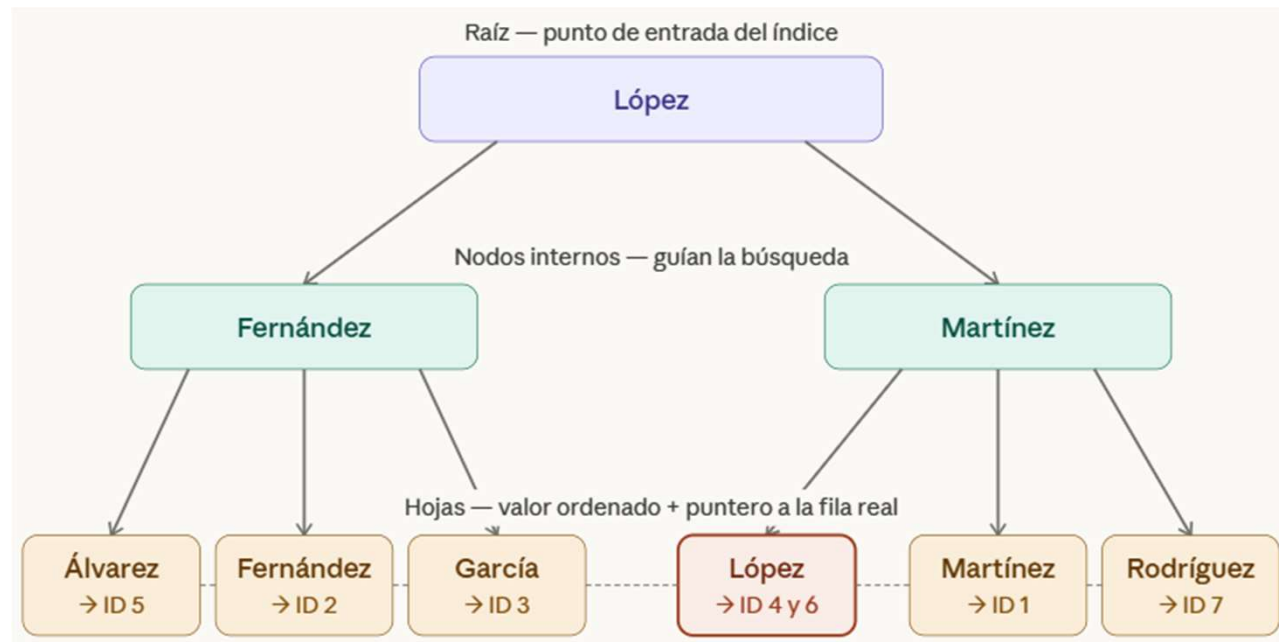
Índices en Base de Datos

Índices B-Tree

Lo primero que hace es **ordenar los valores** alfabéticamente:

Álvarez → Fernández → García → López → López → Martínez → Rodríguez

ID	Apellido	...
1	Martínez	
2	Fernández	
3	García	
4	López	
5	Álvarez	
6	López	
7	Rodríguez	



Índices en Base de Datos

Índices B-Tree

Regla para la navegación del B-Tree

En cada nodo interno, el valor actúa como **divisor**:

- $< \text{valor} \rightarrow$ izquierda
- $\geq \text{valor} \rightarrow$ derecha

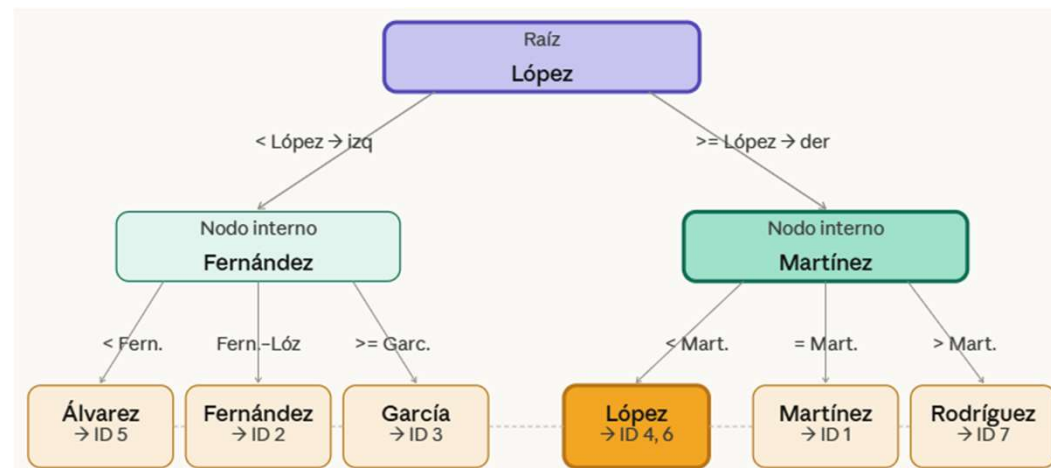
Índices en Base de Datos

Índices B-Tree

¿Cómo busca el SGBD¹ **WHERE** apellido = 'Lopez'?

La búsqueda son **solo 3 pasos**:

- 1. Raíz:** ¿López = López? → ir a la rama derecha
- 2. Interno:** ¿López < Martínez? → ir a la hoja izquierda de esa rama
- 3. Hoja:** Encontrado López → devuelve ID 4 y ID 6 ✓



[¿Cómo busca?](#)

¹ Sistema de Gestión de Base d Datos

Índices en Base de Datos

Índices B-Tree

¿Cuándo usar B-Tree?

Usa B-Tree	Evita B-Tree
Búsqueda por igualdad WHERE apellido = 'López'	Columnas de baja cardinalidad genero, estado (solo 2-3 valores)
Rangos y comparaciones BETWEEN, <, >, <=, >=	Búsqueda de texto libre LIKE '%López%' (% al inicio)
Ordenamiento ORDER BY, GROUP BY	Datos geoespaciales coordenadas, polígonos → GiST
Claves primarias y foráneas JOIN entre tablas	Datos JSON / arrays columnas jsonb → GIN
Prefijos de texto LIKE 'López%' (% solo al final)	Tablas pequeñas < 1000 filas: full scan es más rápido
Columnas con alta cardinalidad email, DNI, id (muchos valores únicos)	Writes muy frecuentes INSERT/UPDATE masivos en tiempo real

Índices en Base de Datos

Índices B-Tree

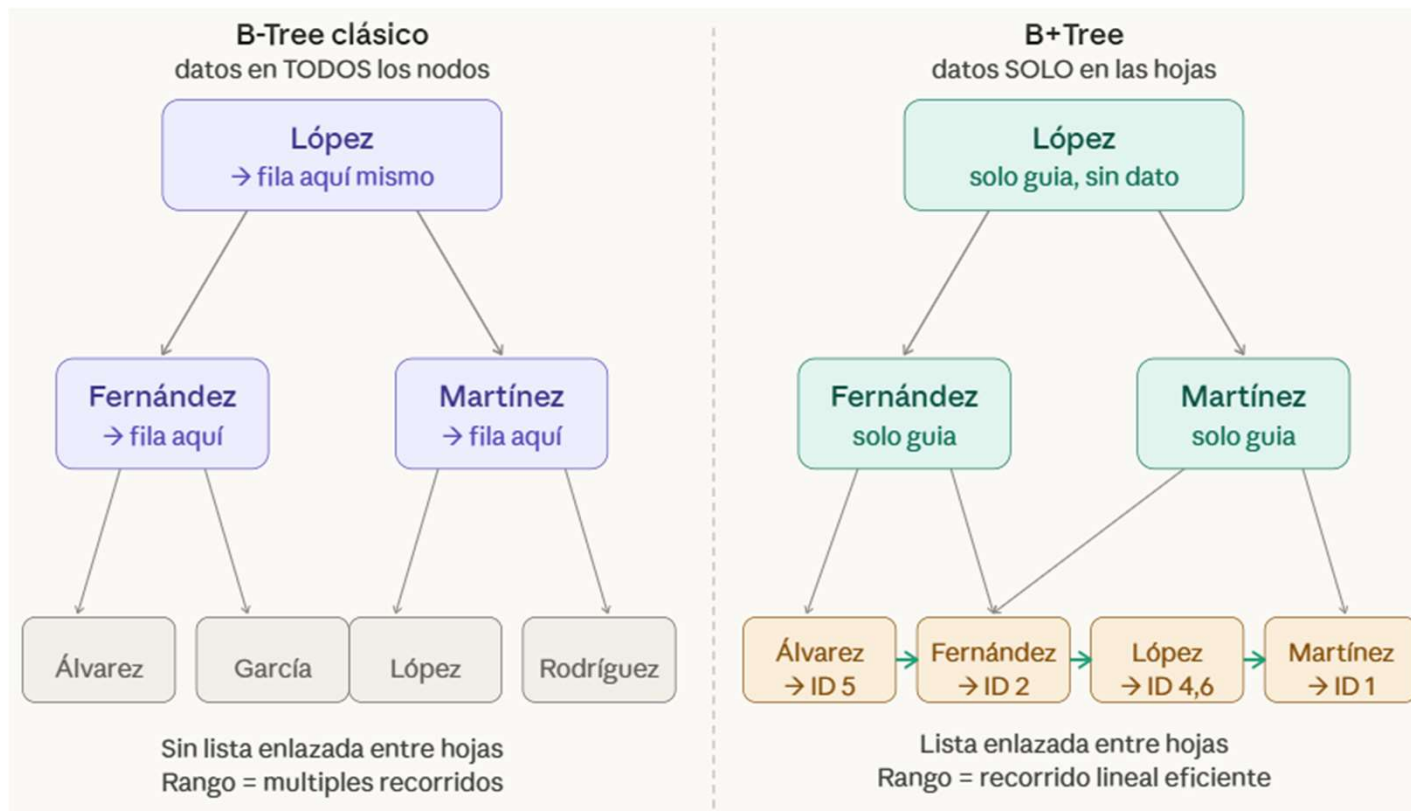
Variantes importantes

- B-Tree: Las **claves** se almacenan en nodos internos y hojas (más común en SGBD)
- B+Tree : Las **claves** con punteros a los datos están **solo en las hojas**, lo que da mayor factor de ocupación y menos reequilibrios (menos ajustes internos al crecer o reducirse)

Índices en Base de Datos

Índices B-Tree

Comparación de B-Tree y B+Tree



Índices en Base de Datos

Índices B-Tree

Ejemplo de cómo se crean los índice B-Tree en PostgreSQL

-- Índice simple

```
CREATE INDEX idx_customers_lastname  
ON customers(lastname);
```

-- Índice compuesto

```
CREATE INDEX idx_orders_customer_date  
ON orders(customer_id, order_date);
```

-- Índice único

```
CREATE UNIQUE INDEX idx_users_email  
ON users(email);
```

Índices en Base de Datos

Índices B-Tree

La regla del **LIKE** que siempre confunde

Uno de los errores más comunes que vale remarcar:

El B-Tree **SÍ** puede ayudar: **el prefijo es fijo**

WHERE apellido LIKE 'Ló%' ✓ busca desde "Ló..." en adelante

El B-Tree **NO** puede ayudar: **no sabe por dónde empezar en el árbol**

WHERE apellido LIKE '%pez' ✗ full scan igual

WHERE apellido LIKE '%ópe%' ✗ full scan igual los errores

Con 'Ló%' sabe exactamente en qué parte del árbol arrancar.

Con '%pez' tendría que revisar todas las hojas de todas formas.

Índices en Base de Datos

Índices B-Tree

Verificación y análisis

Después de crear el índice, puedes verificar su uso con:

EXPLAIN ANALYZE

SELECT * **FROM** empleados **WHERE** apellido = 'Gómez'

Esto te mostrará si el plan de ejecución utiliza el índice **idx_apellido**.

Índices en Base de Datos

Índices Hash

Índices en Base de Datos

Índices **Hash**

Características

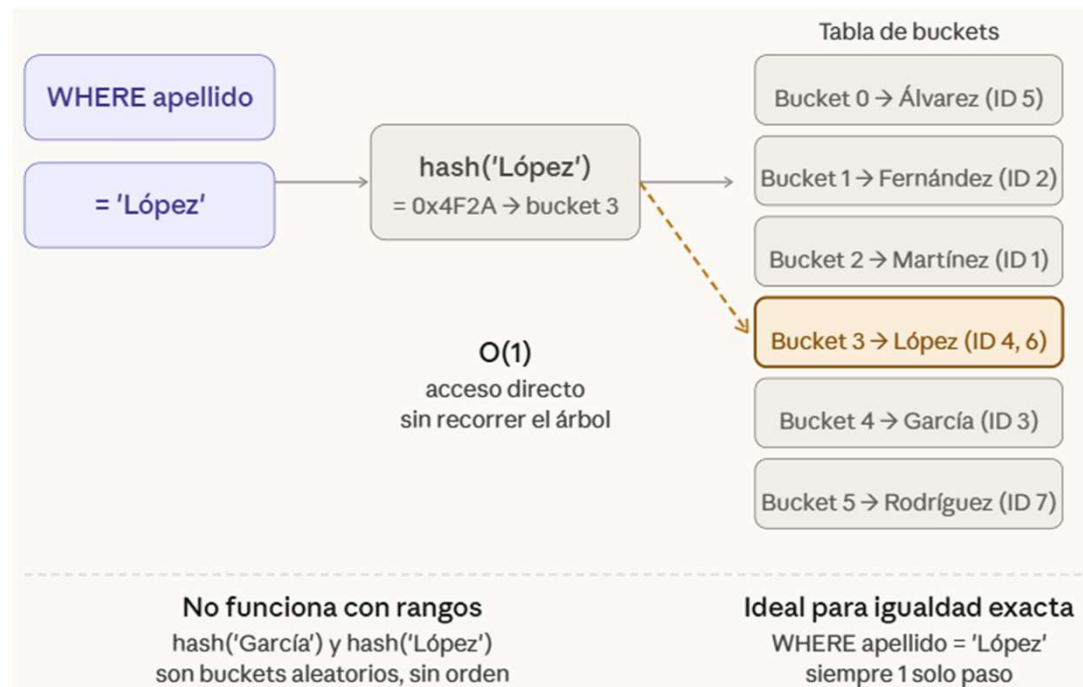
- Basados en funciones hash (transforman la clave en dirección de memoria)
- Extremadamente rápidos para búsquedas exactas
- No mantienen el orden de los datos
- No sirven para búsquedas por rango

Índices en Base de Datos

Índices Hash

Funcionamiento

1. **Hash:** Aplica función hash a la clave para determinar ubicación
2. **Colisiones:** Maneja colisiones mediante técnicas como encadenamiento o sondeo lineal
3. **Acceso directo:** Permite localización directa del registro



Índices en Base de Datos

Índices Hash

Ejemplo de cómo se crean los índice Hash en PostgreSQL

```
CREATE INDEX idx_products_code_hash  
ON products USING HASH (product_code);
```

Índices en Base de Datos

Índices Hash

¿Cuándo usar un índice hash?

Escenario	Justificación
WHERE id = 12345	Búsqueda exacta, acceso directo
WHERE email='usuario@dominio.com'	Clave única, búsqueda exacta
WHERE clave IN ('A123', 'B456')	Búsqueda por igualdad múltiple
JOIN ON usuario_id= cliente_id	Operación de JOIN por igualdad
Tablas temporales o de caché	Acceso rápido, sin necesidad de orden

Índices en Base de Datos

Índices Hash

¿Cuándo **NO** usar un índice hash?

Escenario	Justificación
WHERE fecha BETWEEN '2026-01-01' AND '2026-12-31'	El índice hash no soporta comparaciones de rango.
ORDER BY nombre	Hash no mantiene orden
LIKE 'Juan%'	No compatible con patrones
Rendimiento depende de la calidad de la función hash	Una mala función hash puede generar demasiadas colisiones y degradar el rendimiento.
Algunos SGBD no los soportan nativamente	Ejemplo: en PostgreSQL estuvieron deshabilitados por años y recién se estabilizaron en versiones recientes.
Problemas con alta tasa de colisiones	Si muchas claves generan el mismo hash, el acceso deja de ser $O(1)$ y se convierte en búsquedas lineales dentro del bucket.

Índices en Base de Datos

Índices Hash

¿Qué es una colisión?

Ocurre cuando dos valores distintos producen el **mismo bucket**.

Por ejemplo:

`hash('García') = bucket 2`

`hash('Martínez') = bucket 2` ← **colisión!**

Existen dos posibles soluciones

Índices en Base de Datos

Índices Hash - Colisiones

1. Encadenamiento (Chaining)

Cada bucket tiene una **lista enlazada** de todos los valores que cayeron en el mismo lugar.



Bucket 3: como **López** aparece dos veces (ID 4 y ID 6), el SGBD los encadena

Cuando buscás **WHERE apellido = 'López'**, llega al bucket 3 y recorre la cadena devolviendo ambos.

Índices en Base de Datos

Índices Hash - **Colisiones**

2. Open Addressing (Direccionamiento Abierto)

En lugar de hacer una lista, busca el **siguiente bucket libre** dentro de la misma tabla.

Hay tres variantes:

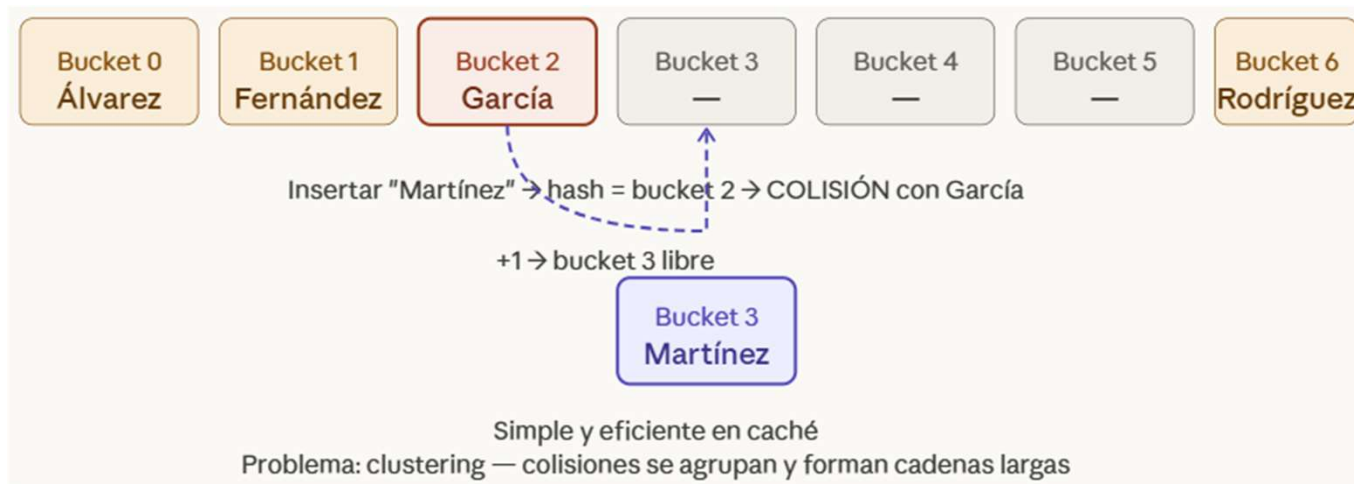
- 1. Sondeo lineal**
- 2. Sondeo cuadrático**
- 3. Hash doble**

Índices en Base de Datos

Índices Hash - Colisiones - **Open Addressing**

1. Sondeo lineal

Ante una colisión, prueba el siguiente bucket: +1, +2, +3... hasta encontrar uno libre.

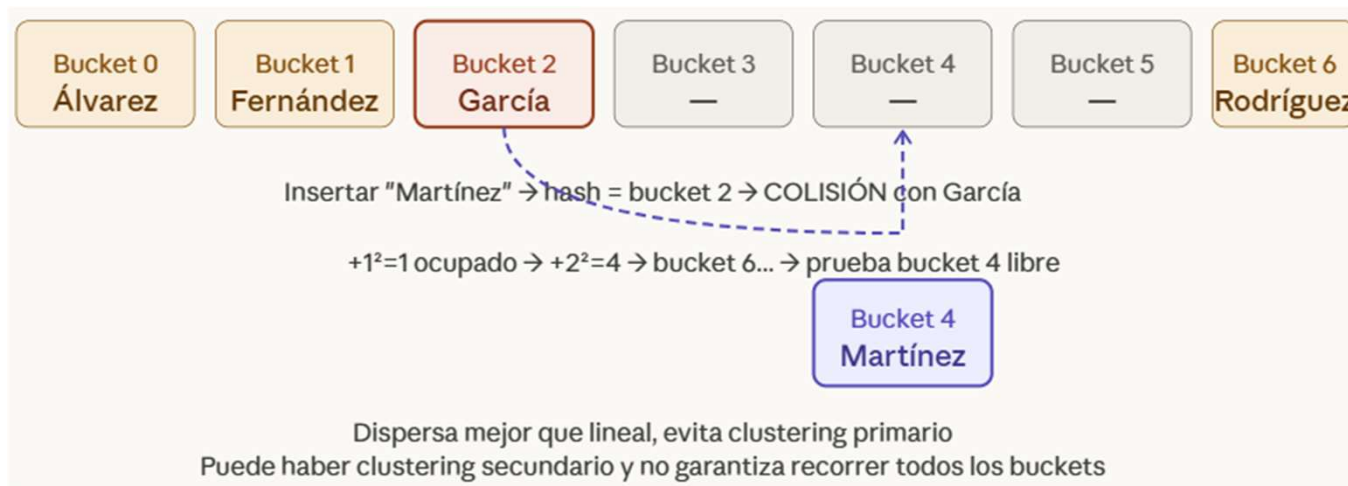


Índices en Base de Datos

Índices Hash - Colisiones - **Open Addressing**

2. Sondeo cuadrático

Ante una colisión, prueba posiciones cuadráticas:
 $+1^2$, $+2^2$, $+3^2$... Dispersa mejor que el lineal.



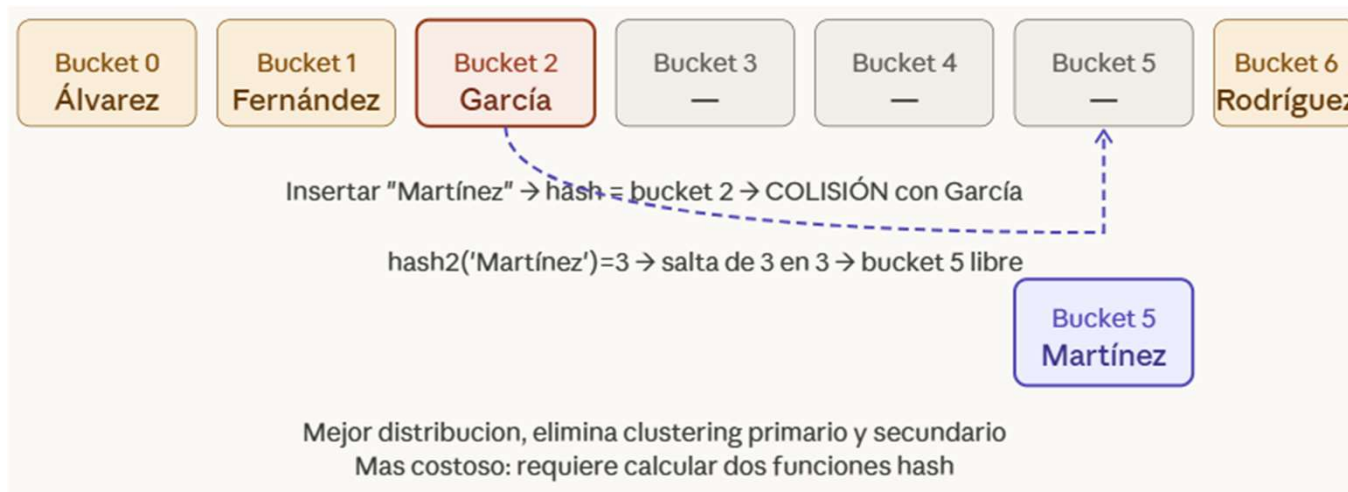
Índices en Base de Datos

Índices Hash - Colisiones - **Open Addressing**

3. Hash doble

Ante una colisión, aplica una segunda función hash para calcular el salto: $\text{bucket} + i \times \text{hash2}(\text{valor})$.

Cada clave tiene su propio patrón de salto.



Índices en Base de Datos

Índices Hash

El factor de carga (el número más importante)

Todos los SGBD monitorizan esto:

factor de carga = elementos insertados / total de buckets

Cuando el factor sube demasiado, las colisiones se disparan y el $O(1)$ se degrada.

Se debe hacer **rehashing**: crean una tabla más grande y redistribuyen todo:

factor < 0.7 → todo bien, pocas colisiones

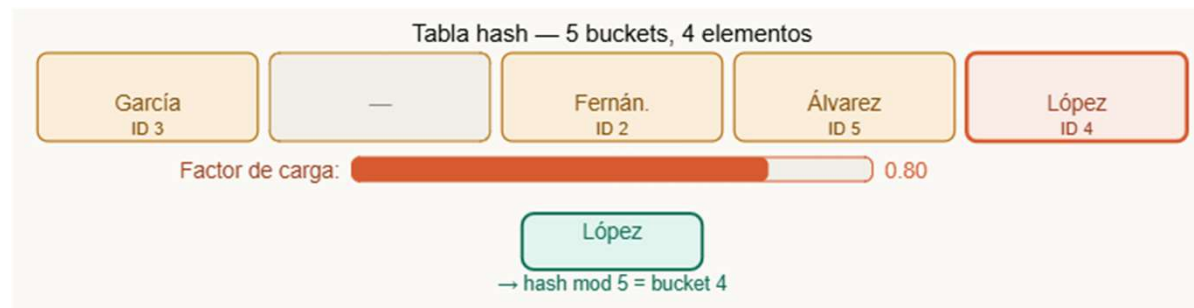
factor > 0.7 → rehashing automático, se duplica la tabla

Índices en Base de Datos

Índices Hash

Rehasing

Insertamos **López** → bucket 4. Factor = $4/5 = 0.80$
supera 0.70 → ¡Rehashing necesario!



Índices en Base de Datos

Índices Hash

Rehasing

se crea una tabla nueva con **11 buckets** (número primo).

Cada elemento se recalcula con hash mod 11.

Factor nuevo = $4/11 = 0.36$

